

H A S H E D I N B Y D E L O I T T E
Engineering Learning & Development

Multi-Agent Systems Engineering

Optional Course Content — Post-Training Extension Material

Document	MAS-TRAINING-002
Audience	Engineers who have completed the 8 required modules
Author	Nabeel Qazi
Owner	Engineering Learning & Development
Version	1.0
Status	Internal Use

Document Control

Document Title	Multi-Agent Systems Engineering — Optional Course Content
When to Use	After completing all 8 required modules (MAS-TRAINING-001)
Companion Docs	MAS-TRAINING-001 (Mandatory Content) · MAS-TRAINING-003 (Assignments) · MAS-TRAINING-004 (Evaluator Guide)
Not Assessed	None of the lessons in this document are required for the 10 graded assignments
Author	Nabeel Qazi
Owner	Engineering Learning & Development

Introduction

This document contains the self-directed extension material for the Multi-Agent Systems Engineering program. None of these lessons are required to pass the 10 graded assignments — they are provided for engineers who want to go deeper into specific frameworks or production runtimes after completing the core curriculum.

Contents of This Document

Section	Topic	Time
Module 8 — Lesson 8.4	Knowledge Graphs for Agent Memory with Neo4j	~3 hours
Module 8 — Lesson 8.5	MemGPT / Letta: LLM-as-OS Memory	~1.5 hours
Module 8 — Lesson 8.6	OpenClaw: Self-Hosted Multi-Agent Memory & Orchestration	~3.5 hours
Optional Module A	OpenAI Agents SDK	~6 hours
Optional Module B	Claude Agent SDK (Anthropic)	~6 hours
Optional Module C	Google ADK and Agent Engine	~12 hours

Recommended Learning Paths

Choose the optional content that best matches your next project:

If you want to...	Start with
Build enterprise agents that reason over structured relationships and graphs	Lesson 8.4 — Neo4j Knowledge Graphs
Deploy long-running autonomous agents with managed memory	Lesson 8.5 — Letta
Self-host a full agent infrastructure stack (gateway + memory + tools)	Lesson 8.6 — OpenClaw
Build OpenAI-native agents with built-in guardrails and sessions	Optional Module A — OpenAI Agents SDK
Build on Claude with the Agent SDK and MCP integration	Optional Module B — Claude Agent SDK
Deploy production agents on Google Cloud with managed runtime	Optional Module C — Google ADK

Module 8 — Optional Extension Lessons

Lessons 8.4, 8.5, and 8.6 extend Module 8 (Agentic Memory). The required lessons 8.1–8.3 and 8.7 are covered in MAS-TRAINING-001.

Lesson 8.4 — Knowledge Graphs for Agent Memory with Neo4j

OPTIONAL LESSON — Self-directed; not required for assessment

Neo4j is the world's leading graph database. Where vector stores capture semantic similarity, knowledge graphs capture structured relationships — who knows whom, what belongs to what, which event caused which outcome. Combining both in a single agent memory layer produces richer, more explainable retrieval than either approach alone.

Why Knowledge Graphs Matter for Agent Memory

Agents that interact with users over time encounter a growing web of entities and relationships. A flat vector store answers 'what is semantically similar to this query?'. A knowledge graph answers 'how are these entities connected?' — and that difference matters in enterprise contexts where relationships, provenance, and multi-hop reasoning are essential.

Learning Outcomes

- Explain why knowledge graphs complement vector stores — relationship-aware retrieval catches connections that similarity search misses.
- Write Cypher queries to store and retrieve structured knowledge from Neo4j.
- Build a knowledge graph from unstructured text using LangChain's LLMGraphTransformer and Neo4j.
- Apply Neo4j's combined vector + graph (GraphRAG) capabilities for hybrid retrieval: semantic search and structured Cypher in the same pipeline.
- Use Neo4jSaver as a LangGraph checkpoint saver for persistent, branch-capable agent memory with time-travel support.
- Compare Neo4j knowledge graph memory to flat vector-store approaches (Mem0, LangMem) and choose the right tool for the use case.

Resources

- [DeepLearning.AI — Knowledge Graphs for RAG \(free, with Neo4j\)](#) (1 hour)
- [DeepLearning.AI — Agentic Knowledge Graph Construction \(free, Neo4j + ADK\)](#) (1 hour)
- [Neo4j — LangChain Integration \(official docs: vector, graph, Cypher, checkpoint\)](#) (1 hour)
- [LangChain — Neo4j integration documentation](#) (30 mins)
- [Neo4j blog — GraphRAG workflow with LangChain and LangGraph](#) (45 mins)
- [Neo4j GraphAcademy — Knowledge Graph and GraphRAG learning path \(free\)](#) (2 hours)
- [Medium — GraphRAG Tutorial: Neo4j + LLMs end-to-end](#) (30 mins)

Lesson 8.5 — MemGPT / Letta: LLM-as-OS Memory

OPTIONAL LESSON — Self-directed; not required for assessment

MemGPT (now Letta) treats the context window like RAM and lets the agent page memory in and out — the LLM-as-OS thesis. Letta's Agent Development Environment (ADE) provides visual memory inspection and is well-suited for long-running autonomous agents.

Learning Outcomes

- Understand the MemGPT thesis: treat the context window like RAM and let the agent page memory in and out.
- Use Letta's Agent Development Environment (ADE) for visual memory inspection.
- Recognize when Letta's full-stack approach fits versus when bolt-on memory (Mem0) is preferable.

Resources

- [Letta documentation](#) (1 hour)
- [Letta forum — agent memory comparison thread](#) (20 mins)

Lesson 8.6 — OpenClaw: Self-Hosted Multi-Agent Memory & Orchestration**OPTIONAL LESSON — Self-directed; not required for assessment**

OpenClaw is an open-source, self-hosted personal AI infrastructure platform: a gateway, runtime, sessions, sandboxed tools, multi-channel deployment, and explicit multi-agent orchestration. Local-first, BYO-API-key, with built-in memory compaction.

Why It's Relevant to Memory

OpenClaw is one of the clearest worked examples of durable agent memory in practice: persistent agents bound to channels, sub-agents that auto-archive when done, per-agent workspaces holding files and conversation history, and memory compaction to keep token costs predictable.

Learning Outcomes

- Configure persistent agents (long-lived) and sub-agents (task-scoped, auto-archiving) in a single OpenClaw gateway.
- Use per-agent workspaces and channel bindings to keep memory and identity isolated.
- Apply OpenClaw's memory-compaction techniques to control token cost on long-running sessions.
- Connect OpenClaw to Claude (API key path) and reason about the cost model for agentic workloads.
- Distinguish OpenClaw's runtime memory from pluggable layers like Mem0 or Zep — and decide when to combine them.

Resources

- [OpenClaw — Multi-Agent Workflows \(official docs\)](#) (1 hour)
- [OpenClaw — Anthropic provider \(auth, caching, 1M context\)](#) (30 mins)
- [OpenClaw architecture deep-dive](#) (30 mins)
- [Multi-agent orchestration guide](#) (30 mins)
- [Meta-Intelligence — OpenClaw Subagents, Agent Teams & Orchestration](#) (30 mins)
- [LumaDock — Running multiple OpenClaw agents in one gateway](#) (45 mins)

Optional Module A — OpenAI Agents SDK

OPTIONAL MODULE — Self-directed; not graded. Recommended for engineers building on this stack.

OpenAI's official multi-agent framework (openai-agents, v0.14+, April 2026). Provider-agnostic via LiteLLM — ships handoffs, guardrails, sessions, tracing, and sandbox agents as primitives. Recommended for engineers building OpenAI-native agent stacks or wanting a leaner alternative to LangChain for OpenAI-centric work.

Estimated time: ~6 hours

Learning Outcomes

- Build agents using the Agent + Runner model: instructions, tools, guardrails, handoffs.
- Apply the agents-as-tools pattern and explicit handoff orchestration.
- Use Sessions for conversation memory and built-in Tracing for observability.
- Configure Sandbox Agents (v0.14+) for long-horizon work with persistent workspaces.
- Build voice agents with gpt-realtime.

Resources

- [OpenAI Agents SDK — official documentation](#) (1.5 hours)
- [GitHub — openai/openai-agents-python \(with examples\)](#) (code)
- [OpenAI — The next evolution of the Agents SDK](#) (15 mins)
- [OpenAI Cookbook — Agents topic](#) (2 hours)
- [OpenAI — Agents API guide](#) (30 mins)

Optional Module B — Claude Agent SDK (Anthropic)

OPTIONAL MODULE — Self-directed; not graded. Recommended for engineers building on this stack.

Anthropic's flagship agent runtime. The SDK bundles a built-in tool loop, file/bash/edit primitives, MCP server integration, and a sandboxed execution model. Recommended for engineers building on Claude — and for any engineer who wants to study how Anthropic itself thinks about agent harnesses.

Client SDK vs. Agent SDK

With the Client SDK you implement the tool loop yourself. With the Agent SDK, Claude handles it — gather context, take action, verify work, repeat. The SDK ships built-in tools (Read, Write, Edit, Bash, Glob), permission allowlists, and disallowedTools for safety.

Estimated time: ~6 hours

Learning Outcomes

- Distinguish the Client SDK from the Agent SDK and know when each applies.
- Apply the gather-context → take-action → verify-work → repeat feedback pattern.
- Use built-in tools, permission modes, and MCP servers.
- Apply permission allowlists, sandboxing, and disallowedTools for safe deployment.
- Deploy via the Python or TypeScript SDKs.

Resources

- [Anthropic — Building agents with the Claude Agent SDK](#) (30 mins)
- [Claude Agent SDK — overview docs](#) (1 hour)
- [GitHub — anthropics/claude-agent-sdk-python](#) (code)
- [GitHub — anthropics/claude-agent-sdk-typescript](#) (code)
- [YouTube — Claude Agent SDK Full Workshop](#) (workshop)

Optional Module C — Google ADK and Agent Engine

OPTIONAL MODULE — Self-directed; not graded. Recommended for engineers building on Google Cloud.

Google's Agent Development Kit (ADK) is a production-grade runtime and SDK for building, deploying, and operating agents on Google Cloud. Recommended for engineers targeting Google Cloud deployments or wanting a managed production agent runtime.

Engineers using AWS or Azure may find the required modules sufficient for most use cases.

Estimated time: ~12 hours

Learning Outcomes

- Install and set up ADK; understand its component model (agents, tools, sessions).
- Build basic-to-advanced agentic workflows using ADK primitives.
- Reason about deployment, scaling, and observability for production agents on Google Cloud.
- Build voice agents using ADK's real-time audio capabilities.

Resources

- [YouTube — Introduction to Google ADK](#) (10 mins)
- [ADK Official Documentation](#) (6–7 hours)
- [YouTube — ADK Masterclass](#) (3.5 hours)
- [YouTube — Building Apps with ADK](#) (1 hour)
- [YouTube — Building Voice Agents with Google ADK](#) (50 mins)
- [Medium — End-to-End ADK Implementation](#) (30 mins)